REQ 3
A: Fleshy Tree Growth
Design 1: Each growth stage is a separate class that extends ground. Growth is shown by replacing the current ground with a new instance of the next evolution of the tree.

Pros	Cons
Meets SRP because each class will own only one stage's behaviour. An example of this includes the Sapling spawning a slime and it has a 25% ticking chance of becoming the sapling.	Violates DRY because canActorEnter() which returns false is repeated across all flora classes rather than inherited rather than inherited from an abstract class of Tree.
Meets OCP because by adding new stages this means that the original ground method does not need to be altered and only new methods are implemented.	
Meets KISS principle because each class is simple and all use the tick() method to read linearly and there are no nested conditionals or switch statements.	

Design 2: There is one FleshyTree class which holds a stage enum and switches between (sprout, sapling and mature) in tick() based on the current stage.

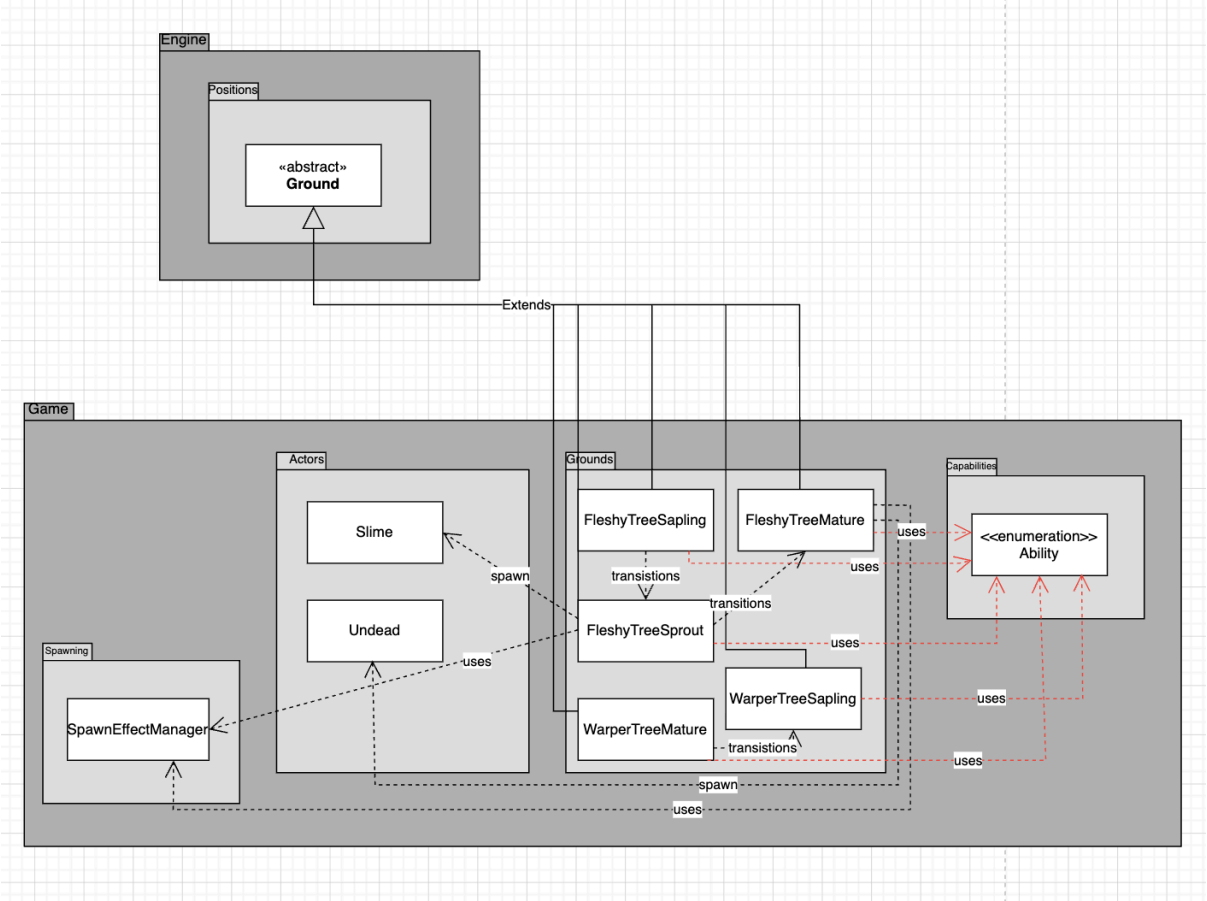
Pros	Cons
DRY is met because canActorEnter() and the field random are defined once. The code does not need to be copied and pasted throughout.	This violates the SRP principle because the class becomes responsible for all three stages and has multiple responsibilities. Tick() will require a new conditional every time a new stage of tree growth is added.
	Violates principle OCP because adding a new stage means you will have to modify the existing class of FleshyTree()

Design 3: Utilising an abstract class called Tree which extends all the different tree types.

Pros	Cons
Meets DRY because canActorEnter() returns false and is not copied and pasted across the different Tree classes.	Violates KISS because adding an extra layer of inheritance for a one line method will make the hierarchy more complicated and harder to track.
Meets OCP because new tree types or the different tree stages can be added by extending Tree class without modifying	LSP for the future if canActorEnter() no longer returns false because actors can now enter the tree to e.g. teleport to another

existing classes such as ground.	dimension then this Tree class becomes a liability and overriding this class contradicts the base class contract.
Meets SRP because each Tree class will have their own function and responsibilities but they all inherit blocking actor entry.	

UML Req 4



The diagram shows the 5 new subclasses of ground which include two lifecycle chains of a “Fleshy Tree” and a “Warper Tree”. In this UML I utilised design 1 (Part A). To an extent it violates DRY principles since canActorEnter() False is copied throughout all tree classes, however, after comparing this to adding in an abstract class which all the fleshy tree and warper tree classes will inherit from, we decided that for future proofing this design we should follow design 1 rather than design 3. In the scenario the spawning of a tree blocks actors from entering that ground tile. However, in future designs it is possible that there may be abilities to teleport to another map or get trapped by the tree, therefore it is important that we do not trap this singular method of canActorEnter() into an abstract class because this abstract class will become redundant if the Actor in the future can actually enter the ground tile.